

Experiment No. 10

Character, Digit, or Face Classification using Convolutional Neural Networks (CNNs)

Name = Harsh Kaushik

CU_ID = CU24250248

Subject = Computer Vision Lab

Branch = B.Tech (CSE)

Aim

To design, train, and evaluate a Convolutional Neural Network (CNN) for character, digit, or face classification using a standard image dataset and analyze its performance using appropriate evaluation metrics.

Purpose

Image classification is an important application of deep learning in computer vision. CNNs automatically learn hierarchical image features and reduce the need for handcrafted feature extraction. In this experiment, the **MNIST handwritten digit dataset** is used to build and evaluate a CNN classifier.

Prerequisites

- Basic Python programming
- Digital Image Processing and Computer Vision concepts
- ANN and CNN concepts
- TensorFlow/Keras, OpenCV, NumPy, Matplotlib
- Google Colab/Jupyter Notebook

Step 1: Import Required Libraries

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import cv2
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.metrics import confusion_matrix, classification_report, precision_score, recall_score, f1_score
import seaborn as sns
from google.colab import files

print("TensorFlow version:", tf.__version__)
print("Libraries imported successfully.")
```

```
TensorFlow version: 2.20.0
Libraries imported successfully.
```

Step 2: Load the MNIST Dataset

MNIST contains grayscale handwritten digit images from **0 to 9**. Each image has a size of 28×28 pixels.

```
(x_train_full, y_train_full), (x_test, y_test) = keras.datasets.mnist.load_data()

print("Training images:", x_train_full.shape)
print("Training labels:", y_train_full.shape)
print("Testing images:", x_test.shape)
print("Testing labels:", y_test.shape)
```

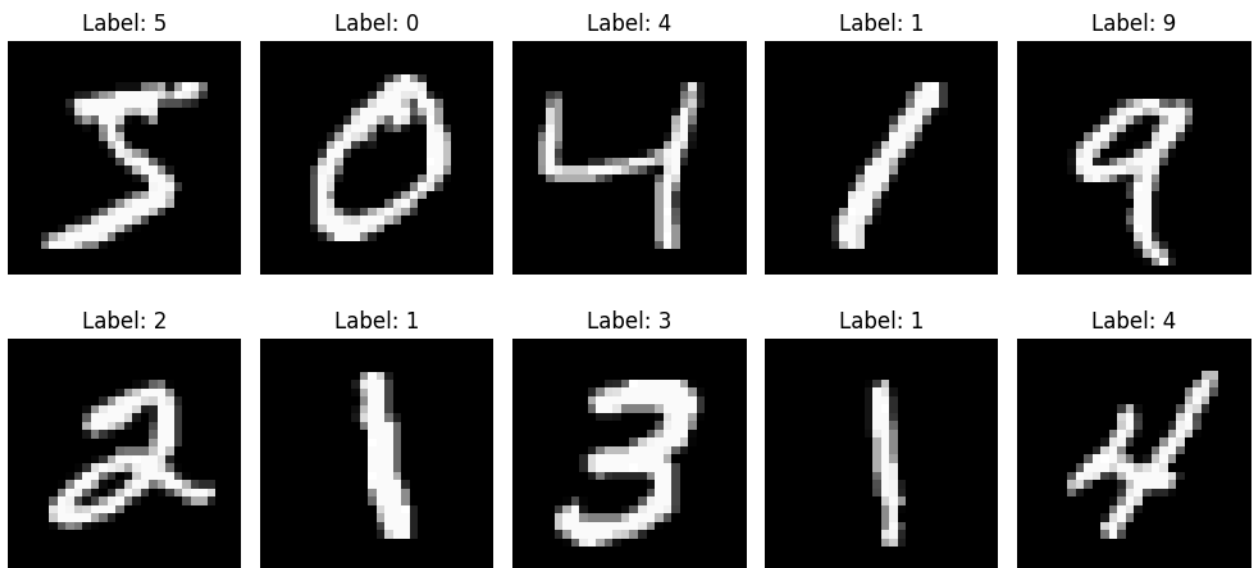
```
Training images: (60000, 28, 28)
Training labels: (60000,)
Testing images: (10000, 28, 28)
Testing labels: (10000,)
```

Step 3: Visualize Sample Images

```
plt.figure(figsize=(10, 5))

for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_train_full[i], cmap="gray")
    plt.title(f"Label: {y_train_full[i]}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```



Step 4: Image Preprocessing

The pixel values originally range from 0 to 255. Normalization scales them to the range 0 to 1. The CNN also expects a channel dimension, so each image is reshaped from 28×28 to 28×28×1.

The original training data is divided into training and validation sets.

```
x_train = x_train_full[:54000]
y_train = y_train_full[:54000]

x_val = x_train_full[54000:]
y_val = y_train_full[54000:]

x_train = x_train.astype("float32") / 255.0
x_val = x_val.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

x_train = np.expand_dims(x_train, axis=-1)
x_val = np.expand_dims(x_val, axis=-1)
x_test = np.expand_dims(x_test, axis=-1)

print("Training set:", x_train.shape)
print("Validation set:", x_val.shape)
print("Testing set:", x_test.shape)
```

```
Training set: (54000, 28, 28, 1)
Validation set: (6000, 28, 28, 1)
Testing set: (10000, 28, 28, 1)
```

Step 5: Design the CNN Architecture

The CNN contains:

- Convolutional layers for feature extraction
- ReLU activation for non-linearity
- MaxPooling layers for dimensionality reduction
- Flatten layer to convert feature maps into a vector
- Fully connected Dense layer for classification
- Softmax output layer for the 10 digit classes

```

model = keras.Sequential([
    layers.Input(shape=(28, 28, 1)),
    layers.Conv2D(32, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(128, activation="relu"),
    layers.Dropout(0.3),
    layers.Dense(10, activation="softmax")
])

model.summary()

```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
conv2d_4 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_4 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_5 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_5 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten_2 (Flatten)	(None, 1600)	0
dense_4 (Dense)	(None, 128)	204,928
dropout_2 (Dropout)	(None, 128)	0
dense_5 (Dense)	(None, 10)	1,290

Total params: 225,034 (879.04 KB)

Trainable params: 225,034 (879.04 KB)

Step 6: Configure the Model

Optimizer: Adam

Loss: Sparse Categorical Crossentropy

Metric: Accuracy

Batch Size: 128

Epochs: 5

Adam is used because it generally provides efficient gradient-based optimization for neural networks.

```

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

print("Model compiled successfully.")

```

Model compiled successfully.

Step 7: Train the CNN

The validation set is monitored during training to observe how well the model generalizes to unseen data.

```

history = model.fit(
    x_train,
    y_train,
    validation_data=(x_val, y_val),
    epochs=5,
    batch_size=128,
    verbose=1
)

```

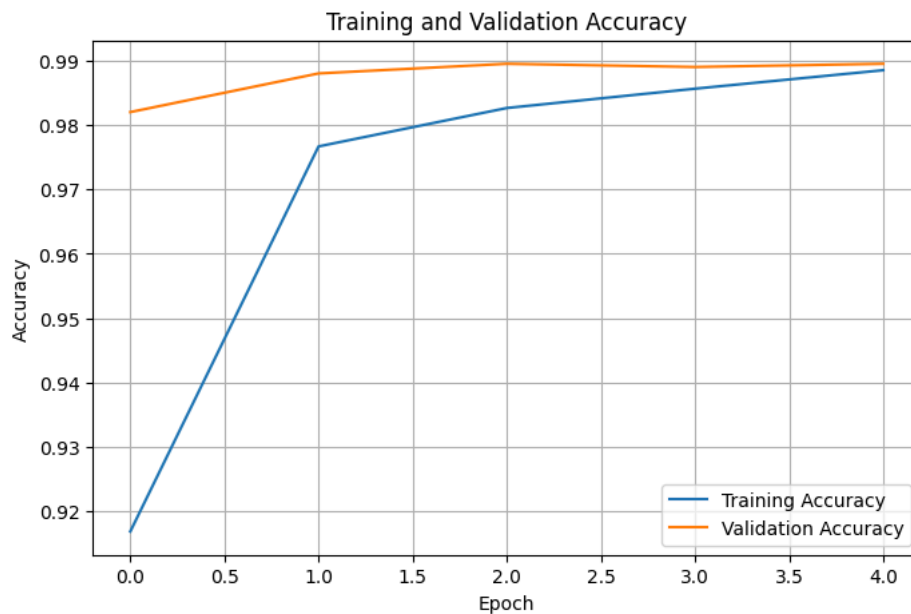
```

Epoch 1/5
422/422 ————— 43s 99ms/step - accuracy: 0.9169 - loss: 0.2738 - val_accuracy: 0.9820 - val_loss: 0.0621
Epoch 2/5
422/422 ————— 80s 93ms/step - accuracy: 0.9767 - loss: 0.0767 - val_accuracy: 0.9880 - val_loss: 0.0449
Epoch 3/5
422/422 ————— 41s 93ms/step - accuracy: 0.9826 - loss: 0.0561 - val_accuracy: 0.9895 - val_loss: 0.0407
Epoch 4/5
422/422 ————— 41s 93ms/step - accuracy: 0.9856 - loss: 0.0458 - val_accuracy: 0.9890 - val_loss: 0.0447
Epoch 5/5
422/422 ————— 40s 94ms/step - accuracy: 0.9885 - loss: 0.0370 - val_accuracy: 0.9895 - val_loss: 0.0387

```

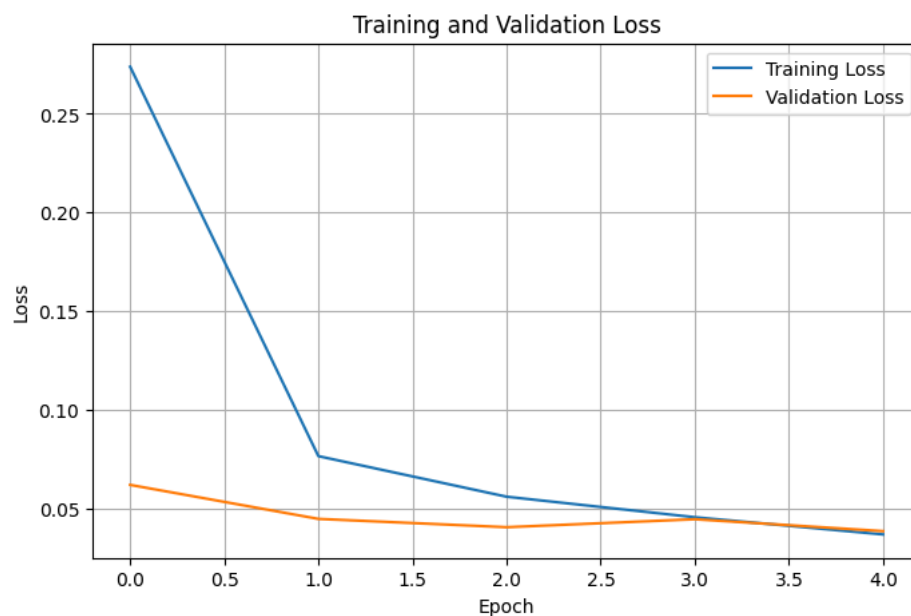
✓ Step 8: Plot Training and Validation Accuracy

```
plt.figure(figsize=(8, 5))
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Training and Validation Accuracy")
plt.legend()
plt.grid(True)
plt.show()
```



✓ Step 9: Plot Training and Validation Loss

```
plt.figure(figsize=(8, 5))
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training and Validation Loss")
plt.legend()
plt.grid(True)
plt.show()
```



Step 10: Evaluate the Model on the Test Dataset

```
test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)

print("Test Loss:", round(test_loss, 4))
print("Test Accuracy:", round(test_accuracy, 4))
print("Test Accuracy (%)ate", round(test_accuracy * 100, 2))
```

```
Test Loss: 0.0287
Test Accuracy: 0.9896
Test Accuracy (%): 98.96
```

Step 11: Generate Predictions

The model outputs probabilities for all ten digit classes. The class with the highest probability is selected as the predicted digit.

```
probabilities = model.predict(x_test, verbose=0)
y_pred = np.argmax(probabilities, axis=1)

print("Predictions generated successfully.")
print("First 20 actual labels: ", y_test[:20])
print("First 20 predicted labels:", y_pred[:20])
```

```
Predictions generated successfully.
First 20 actual labels: [7 2 1 0 4 1 4 9 5 9 0 6 9 0 1 5 9 7 3 4]
First 20 predicted labels: [7 2 1 0 4 1 4 9 5 9 0 6 9 0 1 5 9 7 3 4]
```

Step 12: Calculate Accuracy, Precision, Recall, and F1-Score

For multi-class classification, macro averaging gives equal importance to each digit class.

```
precision = precision_score(y_test, y_pred, average="macro")
recall = recall_score(y_test, y_pred, average="macro")
f1 = f1_score(y_test, y_pred, average="macro")

print("Accuracy:", round(test_accuracy, 4))
print("Precision (Macro):", round(precision, 4))
print("Recall (Macro):", round(recall, 4))
print("F1-Score (Macro):", round(f1, 4))

print("\nDetailed Classification Report:")
print(classification_report(y_test, y_pred, digits=4))
```

```
Accuracy: 0.9896
Precision (Macro): 0.9895
Recall (Macro): 0.9895
F1-Score (Macro): 0.9895
```

```
Detailed Classification Report:
              precision    recall  f1-score   support

0               0.9899       0.9959       0.9929         980
1               0.9965       0.9921       0.9943        1135
2               0.9894       0.9903       0.9898        1032
3               0.9852       0.9911       0.9882        1010
4               0.9949       0.9888       0.9918         982
5               0.9833       0.9877       0.9855         892
6               0.9875       0.9906       0.9891         958
7               0.9855       0.9912       0.9884        1028
8               0.9958       0.9815       0.9886         974
9               0.9871       0.9861       0.9866        1009

 accuracy               0.9896         10000
 macro avg              0.9895         10000
weighted avg              0.9896         10000
```

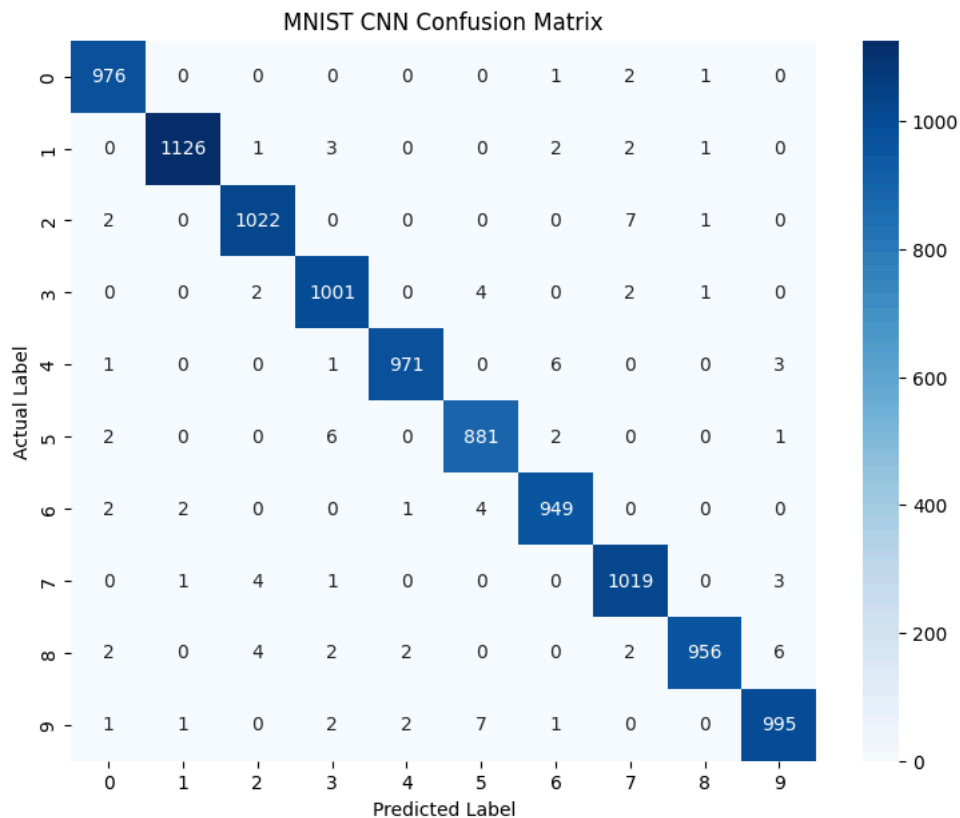
Step 13: Confusion Matrix

The confusion matrix shows how many samples from each actual digit class were predicted as each class. Diagonal values represent correct predictions, while off-diagonal values represent misclassifications.

```
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(9, 7))
```

```
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("MNIST CNN Confusion Matrix")
plt.show()
```



Step 14: Display Correct and Misclassified Predictions

```
correct_indices = np.where(y_pred == y_test)[0]
wrong_indices = np.where(y_pred != y_test)[0]

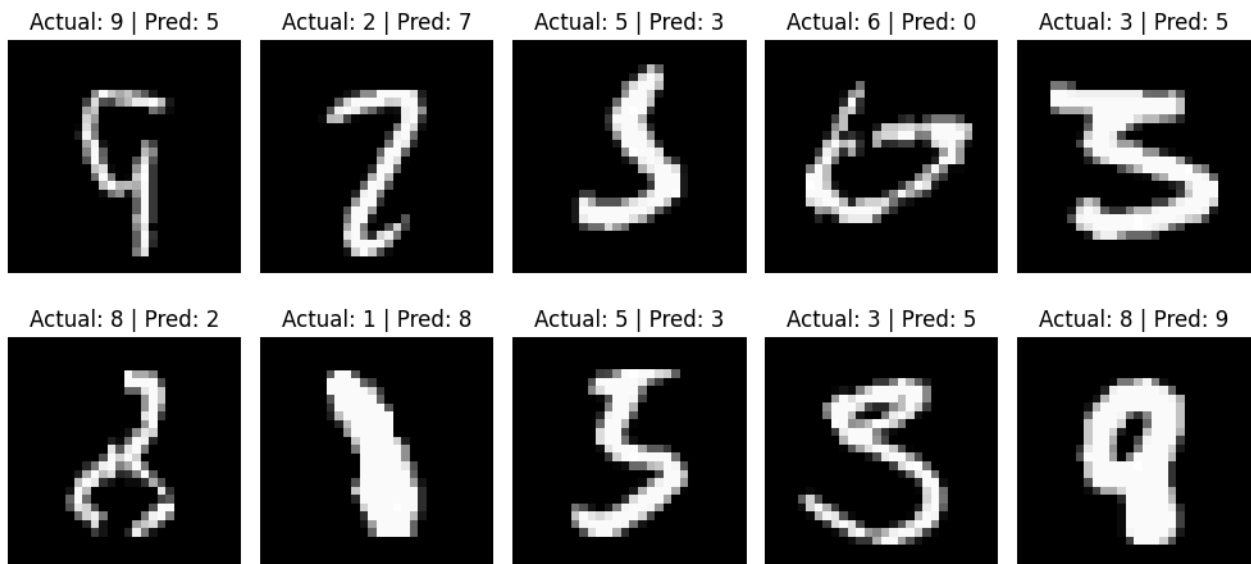
print("Correct predictions:", len(correct_indices))
print("Misclassified predictions:", len(wrong_indices))

plt.figure(figsize=(10, 5))

for i, index in enumerate(wrong_indices[:10]):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_test[index].squeeze(), cmap="gray")
    plt.title(f"Actual: {y_test[index]} | Pred: {y_pred[index]}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```

Correct predictions: 9896
Misclassified predictions: 104



Step 15: Predict an Unseen/Test Image

The following cell selects a test image and displays its actual label, predicted label, and confidence.

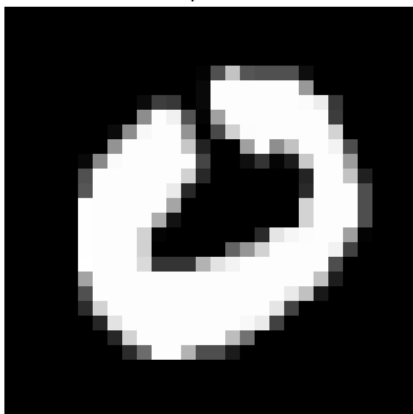
```
sample_index = 25

sample_image = x_test[sample_index]
sample_prediction = model.predict(np.expand_dims(sample_image, axis=0), verbose=0)[0]
predicted_digit = np.argmax(sample_prediction)
confidence = np.max(sample_prediction)

plt.figure(figsize=(4, 4))
plt.imshow(sample_image.squeeze(), cmap="gray")
plt.title(f"Actual: {y_test[sample_index]} | Predicted: {predicted_digit}")
plt.axis("off")
plt.show()

print("Actual digit:", y_test[sample_index])
print("Predicted digit:", predicted_digit)
print("Confidence:", round(float(confidence), 4))
```

Actual: 0 | Predicted: 0



Actual digit: 0
Predicted digit: 0
Confidence: 0.9999

Step 16: Test on a User-Uploaded Digit Image

Upload a clear image containing a single handwritten digit. The image is converted to grayscale, resized to 28×28, normalized, and passed to the trained CNN.

Note: MNIST uses white digits on a black background. If your uploaded digit has a black digit on a white background, the preprocessing below automatically checks the average intensity and inverts the image when appropriate.

```
uploaded = files.upload()

if len(uploaded) > 0:
    upload_name = next(iter(uploaded))
    uploaded_bytes = np.frombuffer(uploaded[upload_name], np.uint8)
    uploaded_image = cv2.imdecode(uploaded_bytes, cv2.IMREAD_GRAYSCALE)

    if uploaded_image is None:
        raise ValueError("Uploaded image could not be read.")

    digit_image = cv2.resize(uploaded_image, (28, 28))

    if np.mean(digit_image) > 127:
        digit_image = 255 - digit_image

    digit_image = digit_image.astype("float32") / 255.0
    digit_input = np.expand_dims(digit_image, axis=(0, -1))

    digit_probabilities = model.predict(digit_input, verbose=0)[0]
    digit_prediction = np.argmax(digit_probabilities)
    digit_confidence = np.max(digit_probabilities)

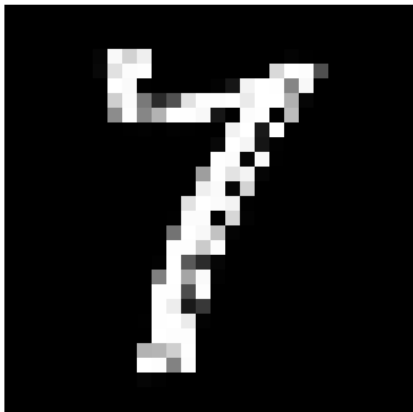
    plt.figure(figsize=(4, 4))
    plt.imshow(digit_image, cmap="gray")
    plt.title(f"Predicted Digit: {digit_prediction}")
    plt.axis("off")
    plt.show()

    print("Predicted digit:", digit_prediction)
    print("Confidence:", round(float(digit_confidence), 4))
```

Choose Files images.jpg

images.jpg(image/jpeg) - 5874 bytes, last modified: 9/22/2026 - 100% done
Saving images.jpg to images.jpg

Predicted Digit: 7



Predicted digit: 7
Confidence: 0.8474

Step 17: Performance Summary

```
metrics = {
    "Accuracy": test_accuracy,
    "Precision": precision,
    "Recall": recall,
    "F1-Score": f1
}

metrics_df = pd.DataFrame(metrics.items(), columns=["Metric", "Value"])
metrics_df["Percentage"] = metrics_df["Value"] * 100

display(metrics_df.round(4))
```

	Metric	Value	Percentage
0	Accuracy	0.9896	98.9600
1	Precision	0.9895	98.9498
2	Recall	0.9895	98.9535
3	F1-Score	0.9895	98.9507

Observations

1. The CNN automatically learns useful digit features such as edges, curves, and stroke patterns.
2. Normalization makes optimization more stable and efficient.
3. Convolutional and pooling layers progressively learn higher-level visual features.
4. The validation curves help identify overfitting or underfitting.
5. The confusion matrix shows which digit classes are most frequently confused.
6. Precision, recall, and F1-score provide more information than accuracy alone.
7. Misclassifications can occur when handwritten digits have similar shapes or unusual writing styles.
8. Data augmentation, better architectures, and hyperparameter tuning can further improve generalization.

Questions and Answers

1. What is image classification? How does it differ from object detection and image segmentation?

Image classification assigns one or more class labels to an image or image region. Object detection identifies objects, their classes, and their locations using bounding boxes. Image segmentation assigns labels to pixels or regions and provides detailed object boundaries.

2. Explain the architecture and working principle of a CNN.

A CNN generally contains convolutional layers, activation functions, pooling layers, and fully connected layers. Convolutional filters learn local patterns such as edges and textures. Pooling reduces spatial dimensions. Deeper layers learn higher-level features. Fully connected layers use these features for final classification, and an output layer produces class probabilities.

3. What is the role of Convolutional, Pooling, and Fully Connected Layers?

Convolutional layers extract spatial features. **Pooling layers** reduce feature-map size and computational cost while retaining important information. **Fully connected layers** combine learned features and perform final classification.

4. Why is image normalization performed before training?

Normalization scales pixel values to a smaller range, commonly 0–1. It improves numerical stability, helps gradient-based optimization converge efficiently, and generally makes neural-network training more stable.

5. Explain ReLU and Softmax.

ReLU (Rectified Linear Unit) is commonly used in hidden layers and is defined as $\max(0, x)$. It introduces non-linearity and helps neural networks learn complex patterns. **Softmax** converts final-layer scores into probabilities whose sum is 1, making it suitable for multi-class classification.

6. What is a Confusion Matrix?

A confusion matrix is a table comparing actual and predicted classes. Correct predictions appear on the diagonal, while incorrect predictions appear off the diagonal. It helps identify which classes are being confused.

7. Differentiate Accuracy, Precision, Recall, and F1-Score.

Accuracy = correct predictions / total predictions.

Precision = $TP / (TP + FP)$, measuring how many predicted positives are actually positive.

Recall = $TP / (TP + FN)$, measuring how many actual positives were detected.

F1-Score = $2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$, providing a balance between precision and recall.

For example, if a classifier predicts 10 samples as digit 5 and 8 are actually 5, precision for digit 5 is $8/10 = 0.80$.

8. Compare SIFT/HOG + SVM with CNN-based classification.

SIFT/HOG + SVM uses manually designed or engineered features followed by a traditional classifier. CNNs learn features and the classifier jointly from training images. CNNs can learn more complex hierarchical representations but usually require more data and

computational resources.

9. Mention five real-world applications of CNN-based image classification.

1. Handwritten digit and character recognition
2. Face and biometric recognition
3. Medical image classification
4. Defect classification in manufacturing
5. Image-based document and product classification

10. How can Data Augmentation, Transfer Learning, and Hyperparameter Tuning improve performance?

Data Augmentation creates varied training samples through operations such as rotation, shifting, cropping, or flipping, improving generalization. **Transfer Learning** starts from a model trained on a large dataset and adapts it to a new task, reducing training requirements. **Hyperparameter Tuning** searches for suitable learning rates, batch sizes, architectures, dropout values, and training epochs to improve validation performance.

Result